

MIT

Academy of
Engineering

School of Computer Engineering

Computer Graphics Mini Project (Review 1).

2D SPACE SHOOTER

Guide:
Mr/Mrs/Dr. Madhavi
Nimkar

Students Name:
1. Kartik Ambhore(10)
2. Soham More (14)
3. Shreyas Agre (23)

Index

- **Introduction**
- **Abstract(Aim of your project in five to six lines)**
- **Specify Objects involved in project.**
- **Software/Hardware Requirement**
- **Proposed System**
- **Implementations details (description of various functions used in project)**
- **Design**
- **Demonstration**
- **References**

Introduction

- A 2D Space Shooter game developed using C++ and OpenGL.
- Demonstrates 2D transformations, animation, rendering, and keyboard interaction.
- Includes player, enemies, bullets, and collision detection.
- Combines Computer Graphics concepts to create an interactive game environment.



Fig. 1: Conceptual representation of the 2D Space Shooter game environment showing the player spaceship, enemy spaceships, projectiles, and space background.

Courtesy of: Authors (AI-generated)

Problem statement

- Develop an interactive 2D Space Shooter game using OpenGL that demonstrates fundamental Computer Graphics concepts such as object rendering, transformations, animation, keyboard interaction, and collision detection.

Define Objects



Player Spaceship — controlled by the user.



Enemy Spaceships — move toward the player.



Bullets — fired by the player to destroy enemies.



Explosion — visual effect after collision.

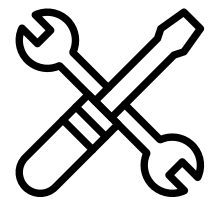


Stars / Space Background — creates the game environment.



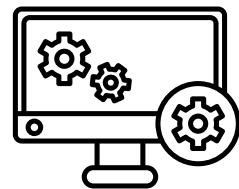
Lives & Score — represents the game state.

Hardware/Software Requirement



Hardware

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard
- Basic graphics support



Software

- C++
- OpenGL
- FreeGLUT
- Visual Studio / VS Code /CodeBlocks
- Windows / Linux OS

Proposed System

- The proposed system is an interactive 2D Space Shooter game developed using C++ and OpenGL.



Key Components:

- *Player spaceship control*
- *Enemy and bullet animation*
- *Collision detection*
- *Score and lives management*
- *Real-time OpenGL rendering*

The system takes keyboard input from the user, updates the positions of game objects, checks for collisions, updates the game state, and continuously renders the updated scene

Implementation details

- `glutInit()` :
 - Initializes the GLUT library and establishes interaction with the windowing system and OpenGL.
- `glutInitDisplayMode()` :
 - Sets the display mode, including double buffering for smooth animation.
- `glutCreateWindow()` :
 - Creates the OpenGL window and displays the game title.
- `glutInitWindowSize()` :
 - Specifies the size of the game window.
- `glutInitWindowPosition()` :
 - Specifies the initial position of the game window on the screen.
- `glutKeyboardFunc()` :
 - Handles keyboard input for player controls and game actions.

Design

Phases involved in the project:

1. Initialization — Initialize OpenGL and create the game window.
2. Object Creation — Create the player, enemies, bullets, and background.
3. Movement & Animation — Implement player controls and continuous object movement.
4. Collision Detection — Detect collisions between bullets, enemies, and player.
5. Game State — Update score, lives, and game-over conditions.
6. Rendering — Continuously render the updated game scene.

Demonstration

- Game Start — Displays the space environment and player spaceship.
- Player Movement — Spaceship moves using keyboard controls.
- Shooting — Player fires bullets at enemy spaceships.
- Collision — Bullets destroy enemies and update the score.
- Game State — Lives and score are updated during gameplay.
- Game Over — Displayed when all lives are lost.

→ GitHub Link : <https://github.com/kartik210906/Computer-Graphics-Miniproject>

References

- [1] OpenGL Documentation
- [2] FreeGLUT Documentation
- [3] Computer Graphics Course Material
- [4] C++ Documentation
- [5] Tutorials and resources used during project development

Thank You!!
Any questions.